

# Software Implementation Specification (SIS) for Secure Document Management System



Module name : Software Engineering  
Module ID : 5CM505  
Instructor : Porgiotis Andronikos

Contributors :  
Student 1 ID : 100777959  
Student 2 ID : 100775044  
Student 3 ID : 100775046

GitHub Repository: <https://github.com/vtsamourtzis/sdms-techbuild>

## 1. Version Control - Git

The project uses Git for source code management, Git was chosen because it is the industry-standard distributed version control system, supports branching and pull-request workflows, and provides a full commit history that evidences iterative development.

### 1.1 Repository Structure

| Directory / File      | Purpose                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------|
| src/auth/             | Authentication classes: User, Engineer, Manager, Supervisor, MFAProvider, SessionManager |
| src/documents/        | Document domain classes: Document, DocumentLock, DocumentRepository                      |
| src/security/         | Security enforcement: SecurityProxy, AccessControlService                                |
| src/audit/            | Audit logging: AuditLog                                                                  |
| src/reports/          | Report generation: Report                                                                |
| src/index.js          | Entry point: wires all components and runs the demonstration scenario                    |
| tests/unit/           | Unit tests for each class (Jest)                                                         |
| tests/integration/    | Integration tests for the authentication and document upload flows (Jest)                |
| coding-conventions.md | Team coding conventions document                                                         |
| package.json          | Node.js project manifest and test runner configuration                                   |

### 1.2 Commit Strategy

Commits follow a feature-per-commit strategy aligned to the design components:

- Initial commit: project structure, package.json, coding conventions
- feat: implement User hierarchy (User, Engineer, Manager, Supervisor)
- feat: implement MFAProvider with OTP generation and single-use verification
- feat: implement SessionManager with rate limiting and lockout
- feat: implement AuditLog with date-range query and retention purge
- feat: implement Document and DocumentRepository
- feat: implement DocumentLock with Observer pattern notifications
- feat: implement SecurityProxy (Proxy pattern) with validation and encryption
- feat: implement AccessControlService (Strategy pattern)
- feat: implement Report aggregation from AuditLog
- test: unit tests for all classes
- test: integration tests for auth flow and document upload flow
- refactor: fix #isLocked bug - preserve failure counter between lock checks

## 2. Coding Conventions

The team agreed a shared conventions document (coding-conventions.md in the repository root) before writing any code. The key conventions are summarised below.

| Convention           | Rule                                                   | Example                                            |
|----------------------|--------------------------------------------------------|----------------------------------------------------|
| File naming          | kebab-case matching the class name                     | audit-log.js, document-lock.js                     |
| Class naming         | PascalCase                                             | AuditLog, DocumentLock, SecurityProxy              |
| Method naming        | camelCase, verb-first                                  | verifyPassword(), generateCode(), queryByUser()    |
| Constants            | UPPER_SNAKE_CASE                                       | MAX_LOGIN_ATTEMPTS, LOCKOUT_DURATION_MS            |
| Private state        | ES2022 private fields (#)                              | #passwordHash, #sessionToken, #failedAttempts      |
| One class per file   | Single responsibility at file level                    | user.js exports only User                          |
| JSDoc                | All public methods documented with @param and @returns | See source files                                   |
| Error types          | Named error subclasses                                 | AccessDeniedError, ValidationError, RateLimitError |
| Strict mode          | 'use strict' at top of every file                      | Prevents accidental globals                        |
| Dependency injection | Dependencies passed via constructor, never hardcoded   | new SessionManager(mfaProvider, auditLog)          |

## 3. Source Code Examples

### 3.1 Basic Functionality - Authentication (SessionManager)

The `SessionManager.initiateLogin()` method demonstrates the two-step credential check with OTP dispatch:

```
initiateLogin(user, password) {
  if (this.#isLocked(user.userID)) {
    return { success: false, message: "Account temporarily locked." };
  }
  if (!user.verifyPassword(password)) {
    this.#recordFailure(user.userID);
    return { success: false, message: "Invalid credentials." };
  }
  this.mfaProvider.generateCode(user.userID, user.email);
  return { success: true, message: "OTP sent to registered email." };
}
```

This maps directly to UC-2 in the SDS: lock check -> password verify -> OTP dispatch. It supports RE-3 (MFA), RE-4 (single session), and RE-13 (rate limiting).

### 3.2 OOP Principles - Inheritance and Polymorphism (User hierarchy)

The User base class defines the contract; subclasses override `hasPermission()` to implement role-specific policies:

```
// Base class - default deny
class User {
  hasPermission(action) { return false; }
}

// Engineer subclass - overrides permission policy (Polymorphism)
class Engineer extends User {
  hasPermission(action) {
    return ['upload', 'download', 'search', 'reserve'].includes(action);
  }
}
```

`AccessControlService` calls `user.hasPermission(action)` without knowing the concrete type. The correct subclass responds transparently - this is Polymorphism implementing the Strategy pattern.

### 3.3 Design Pattern in Code - Proxy (SecurityProxy)

The Proxy pattern is visible in `SecurityProxy.upload()`: all security checks occur before the real storage call:

```
upload(userID, file, metadata) {
  this.#checkRateLimit(userID); // RE-13
  if (!ALLOWED_MIME_TYPES.has(file.mimeType)) // RE-12
    throw new ValidationError("Type not permitted");
  if (file.sizeBytes > MAX_FILE_SIZE_BYTES) // RE-12
    throw new ValidationError("File too large");
  const sanitised = this.#sanitiseMetadata(metadata); // RE-12
  const encrypted = this.#encryptContent(file.content); // RE-9
  const docID = this.documentRepository.save(...); // delegate
  this.auditLog.record(userID, "UPLOAD_SUCCESS"); // RE-15
  return docID;
}
```

```
}
```

### 3.4 Design Pattern in Code - Observer (DocumentLock)

The Observer pattern is visible in DocumentLock: observers register callbacks, and the #notify() helper calls each one when lock state changes:

```
addObserver(callback) { this.#observers.add(callback); }
removeObserver(callback) { this.#observers.delete(callback); }

#notify(type, userID) {
  const event = { docID: this.docID, type, lockedByUserID: userID };
  for (const observer of this.#observers) {
    try { observer(event); }
    catch (err) { console.error("Observer error:", err.message); }
  }
}
```

The try/catch inside the loop ensures one failing observer does not block others - a defensive design decision verified by the unit test "observer error does not prevent other observers firing".

### 3.5 Comments and Low Coupling / High Cohesion

Comments explain why, not what. Example from AuditLog (high cohesion - single responsibility: logging only):

```
/**
 * Returns a snapshot - callers cannot mutate the internal store.
 * Immutability protects audit log integrity.
 */
getAllEntries() { return [...this.#entries]; }
```

Low coupling example: AccessControlService does not import any concrete User subclass. It calls user.hasPermission(action) on whatever User-typed object it receives. This means adding a new role requires zero changes to AccessControlService.

### 3.6 Refactoring - SessionManager #isLocked Bug Fix

During testing, a bug was found in #isLocked: when lockedUntil was null, the original code fell through to delete the failure record, resetting the counter to zero on every check. The refactoring is documented below:

```
// BEFORE (buggy): deletes record even when no lock is set
#isLocked(userID) {
  const record = this.#failedAttempts.get(userID);
  if (!record) return false;
  if (record.lockedUntil && Date.now() < record.lockedUntil) return true;
  this.#failedAttempts.delete(userID); // BUG: wipes counter!
  return false;
}

// AFTER (fixed): only deletes when an expired lock exists
#isLocked(userID) {
  const record = this.#failedAttempts.get(userID);
  if (!record || !record.lockedUntil) return false; // safe exit
  if (Date.now() < record.lockedUntil) return true;
  this.#failedAttempts.delete(userID); // only on expiry
}
```

```
    return false;  
}
```

The bug was caught by the integration test "account locked after 5 failed attempts", which asserted that the 6th login attempt should fail. The root cause was that the short-circuit condition (record.lockedUntil && ...) evaluates to false when lockedUntil is null, causing execution to fall through to the delete statement. The fix adds an early return when no lock is set, preserving the failure counter.

## 4. Design to Code Mapping

The table below maps every SDS design component to its implementing source file and the corresponding test file, completing the V-Model traceability chain from SRS through to tests.

| SDS Component        | Source File                            | Test File                                      | SRS Requirements Covered  |
|----------------------|----------------------------------------|------------------------------------------------|---------------------------|
| User (abstract)      | src/auth/user.js                       | tests/unit/user.test.js                        | RE-1, RE-2, RE-4          |
| Engineer             | src/auth/engineer.js                   | tests/unit/user.test.js                        | RE-6                      |
| Manager              | src/auth/manager.js                    | tests/unit/user.test.js                        | RE-6, RE-16               |
| Supervisor           | src/auth/supervisor.js                 | tests/unit/user.test.js                        | RE-6, RE-16               |
| MFAProvider          | src/auth/mfa-provider.js               | tests/integration/auth-flow.test.js            | RE-3                      |
| SessionManager       | src/auth/session-manager.js            | tests/integration/auth-flow.test.js            | RE-3, RE-4, RE-13         |
| AuditLog             | src/audit/audit-log.js                 | tests/unit/audit-log.test.js                   | RE-14, RE-15              |
| Document             | src/documents/document.js              | tests/unit/security-proxy.test.js              | RE-7, RE-8                |
| DocumentLock         | src/documents/document-lock.js         | tests/unit/document-lock.test.js               | RE-10                     |
| DocumentRepository   | src/documents/document-repository.js   | tests/integration/document-upload-flow.test.js | RE-7                      |
| SecurityProxy        | src/security/security-proxy.js         | tests/unit/security-proxy.test.js              | RE-9, RE-11, RE-12, RE-13 |
| AccessControlService | src/security/access-control-service.js | tests/integration/document-upload-flow.test.js | RE-6                      |
| Report               | src/reports/report.js                  | tests/integration/document-upload-flow.test.js | RE-16                     |

## 5. Software Test Cases

### 5.1 Unit Tests

Unit tests verify each class in isolation. The full suite is in tests/unit/ and runs with `npx jest tests/unit`.

| Test File              | Class Under Test                    | Test Cases                                                                                                                                                                                                                                                                                                             | Result |
|------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| audit-log.test.js      | AuditLog                            | record() creates entry with fields; unique logIDs; timestamp present; queryByUser() filters; queryByDocument() filters; queryByDateRange() includes/excludes; getAllEntries() returns copy                                                                                                                             | PASS   |
| user.test.js           | User, Engineer, Manager, Supervisor | correct password accepted; wrong password rejected; no plaintext exposure; createSession() issues token; second login replaces first token (RE-4); logout() invalidates; role permissions (Engineer, Manager, Supervisor); toPublicProfile() hides secrets                                                             | PASS   |
| document-lock.test.js  | DocumentLock                        | lock granted when free; lock denied when held by another; re-lock by owner allowed; owner unlocks; non-owner denied; SYSTEM force-release; unlock on free returns failure; LOCKED/UNLOCKED/EXPIRED observer events; removed observer not called; failing observer does not block others; getLockInfo returns null/copy | PASS   |
| security-proxy.test.js | SecurityProxy                       | valid PDF accepted; disallowed MIME rejected; file > 50 MB rejected; file at 50 MB limit accepted; HTML stripped from metadata; 10 uploads blocked on 11th; different users have independent limits; download returns decrypted original; unknown docID throws                                                         | PASS   |

### 5.2 Integration Tests

Integration tests verify that multiple components work together correctly. The full suite is in tests/integration/ and runs with `npx jest tests/integration`.

| Test File                    | Flow Tested                        | Test Cases                                                                                                                                                                                                                                                                    | Result |
|------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| auth-flow.test.js            | UC-2: Full authentication flow     | Happy path: correct password + valid OTP -> session token issued; wrong password -> LOGIN_FAILED audit entry; invalid OTP -> no session; second login replaces first session (RE-4); account locked after 5 failures (RE-13); logout invalidates session + LOGOUT audit entry | PASS   |
| document-upload-flow.test.js | UC-7: Document upload and download | Engineer with valid session uploads successfully; upload recorded in audit log with timestamp (RE-15); download returns decrypted original; supervisor cannot upload (AccessDenied); unauthenticated request denied; rejected file type caught before                         | PASS   |



| Test File | Flow Tested | Test Cases                                 | Result |
|-----------|-------------|--------------------------------------------|--------|
|           |             | repository + UPLOAD_BLOCKED<br>audit entry |        |

Total: 57 tests, 57 passed, 0 failed. Test runner: Jest 29. Coverage collected across all src/modules.

## 6. Software Implementation Team Coordination and Contribution

| Student ID | Contributions                                                                                                                                                                                                                                     |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 100777959  | Architecture of src/ module structure; User hierarchy (user.js, engineer.js, manager.js, supervisor.js); SessionManager; AuditLog; index.js demonstration; integration test for auth flow; refactoring of #isLocked bug; SIS document compilation |
| 100775044  | SecurityProxy implementation (Proxy pattern, validation, sanitisation, rate limiting, encryption simulation); AccessControlService (Strategy pattern); security-proxy unit tests; document-upload integration tests; coding conventions document  |
| 100775046  | Document and DocumentRepository; DocumentLock (Observer pattern) and unit tests; AuditLog unit tests; User hierarchy unit tests; Report; package.json setup; README; cross-checking code against SDS design components                            |

Work was managed via the GitHub repository. Each component was developed on the main branch with descriptive commit messages aligned to the coding conventions. A bug found during integration testing (SessionManager rate limiting) was isolated, root-caused, fixed, and re-verified before marking the test suite complete. AI tools were used to explore alternative implementations of the Observer notification loop; the final defensive try/catch design was the team's own decision.